Web Engineering (ASP.NET Core) Assignment

Assignment: Course Management System API

This assignment focuses on implementing database services, dependency injection, entity relationships, DTO validation, authentication, authorization, and optimized data querying using LINQ in ASP.NET Core with Entity Framework Core.

Important Note About Project Models

You are NOT required to use the exact models described in this assignment. You may design your own models and system idea as long as your project follows a similar structure and includes the same technical requirements. This assignment is intended to serve as the foundation for your final course project, so it is recommended that you design models and a system that you plan to continue developing later in the semester.

Learning Objectives

- Implement entity relationships in Entity Framework Core.
- Use dependency injection with database services.
- Design clean service layers.
- Use DTOs for request and response models.
- Validate incoming requests using DTO validation.
- Implement JWT authentication.
- Protect endpoints using authorization.
- Optimize queries using LINQ Select projections.
- Use AsNoTracking() for read-only queries.

Example Core Entities (Optional)

Entity	Description
Instructor	Represents a teacher responsible for courses
Student	Represents a student enrolled in courses
Course	Represents a university course
InstructorProfile	Stores additional information about an instructor
Enrollment	Represents student enrollment in a course

Required Relationships

- Implement one-to-one relationship between two entities.

- Implement one-to-many relationship between two entities.
- Implement many-to-many relationship between two entities.

Required Services

- Create at least three services that manage your main entities.
- Services must use dependency injection for the database context.

DTO Requirements

- Create DTOs for Create operations.
- Create DTOs for Update operations.
- Create DTOs for Read responses.
- Controllers must NOT return entity models directly.

DTO Validation Requirements

- Use Data Annotation attributes such as Required, MaxLength, MinLength, EmailAddress, and Range.
- Invalid requests must return HTTP 400 responses.
- Validation should occur before performing database operations.

Authentication Requirements

- Implement JWT authentication.
- Create an authentication controller.
- Provide a login endpoint that returns a JWT token.
- Clients must send the token using the Authorization header.

Authorization Requirements

- Protect API endpoints using the Authorize attribute.
- Implement role-based authorization.
- Example roles may include Admin, Instructor, or User.
- Restrict certain endpoints to specific roles.

LINQ Optimization Requirements

- Use Select() to project entities into DTOs.
- Avoid returning entire entities when only partial data is required.

AsNoTracking Requirement

All read-only queries should use AsNoTracking() to improve performance.

Technical Requirements

- Use ASP.NET Core Web API.
- Use Entity Framework Core.
- Use async database calls such as ToListAsync, FirstOrDefaultAsync, and SaveChangesAsync.
- Create and apply EF Core migrations.
- Document endpoints using Swagger.

Bonus (Optional)

- Add a background cron job using Hangfire.
- The job may perform a scheduled task such as cleaning old records, sending reminders, or generating reports.
- Implement refresh tokens for authentication to allow secure session renewal.

Submission Requirements

- Complete source code.
- Database migrations.
- A README file explaining how to run the project.
- The README must include all technologies used in the project with a short description of each.
- The README must also explain why HTTP-only cookies are commonly used as an industry standard for authentication security.
- API endpoint documentation.
- Screenshots from Swagger or Postman demonstrating working endpoints.